

EECS 482

Introduction to operating systems

Lock implementation;
deadlock

Peter Chen
University of Michigan

Lock implementation #5

(multiprocessor, minimal busy waiting)

```
lock() {  
    disable interrupts  
    while (test_and_set(guard)) {}  
    if (status != FREE) {  
        add thread to queue of  
        threads waiting for lock  
  
        switch to next ready thread  
    } else {  
        status = BUSY  
    }  
    guard = 0  
    enable interrupts  
}
```

```
unlock() {  
    disable interrupts  
    while (test_and_set(guard)) {}  
    status = FREE  
    if (any thread is waiting  
        for this lock) {  
        move waiting thread to  
        ready queue  
        status = BUSY  
    }  
  
    guard = 0  
    enable interrupts  
}
```

How long could lock() busy wait?

Switch invariant

Multiprocessor

- ~~Uniprocessor~~ switch invariant
 - When calling switch: interrupts must be disabled and guard must be 1
 - When switch returns: thread may assume interrupts are “still” disabled and guard is “still” 1
 - Remember: before running user code, interrupts must be enabled and guard must have been set to 0

Summary of lock solution

- Use low-level mechanisms to provide mutual exclusion for OS code
 - Disable interrupts
 - Spin lock (for multiprocessors)
- OS provides higher-level mechanism (locks) to provide mutual exclusion for user-level code
 - User code runs with interrupts enabled
 - User code runs with no spin locks held (by that CPU)

Summary of lock solution

- Hard problem: how to atomically add thread to a waiting list and sleep
- How did we achieve this?
 - If there is another thread to run: don't sleep! Instead, switch to other thread (obeying switch invariant)
- But what if there is no other thread to run?
 - Need hardware support to atomically enable interrupts and suspend CPU
 - HLT (x86), WFE (ARM), interrupt_enable_suspend (Project 2)

Project 2

- What is this project about?
 - Implementing thread and synchronization
 - Managing the CPUs as they run threads
- Write and test the project incrementally
 - 1 CPU, no interrupts
 - cpu and thread constructor
 - mutex, cv, join
 - Add support for interrupts
 - Add support for multiple CPUs

Deadlock

Interviewer to EECS 482 student: “*Explain deadlock to me, and we’ll give you the job*”.

EECS 482 student to interviewer: “*Give me the job, and I’ll explain deadlock to you*”.

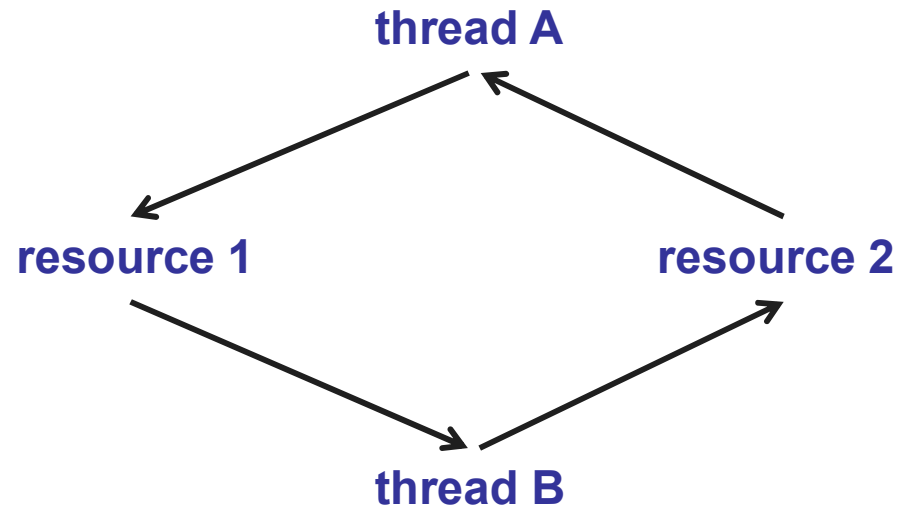
Deadlock

- So far, we have made programs correct by **constraining schedules**
 - Allow only correct interleavings
- But, also possible to **over-constrain** schedules
 - A must happen before B
 - B must happen before A
 - Result is called **deadlock**
- Two types of correctness
 - **Safety**: all actions that occur must be correct
 - **Liveness**: actions must keep occurring. Deadlock violates liveness.

Deadlock

- Resources
 - Things that a thread needs and may need to wait for
 - Examples: locks, disk space, memory, CPU
- Deadlock: a cyclical waiting for resources

Wait-for graph



- Cycle represents a deadlock

Traffic example



Lock example

Thread A

x.lock

y.lock

...

y.unlock

x.unlock

Thread B

y.lock

x.lock

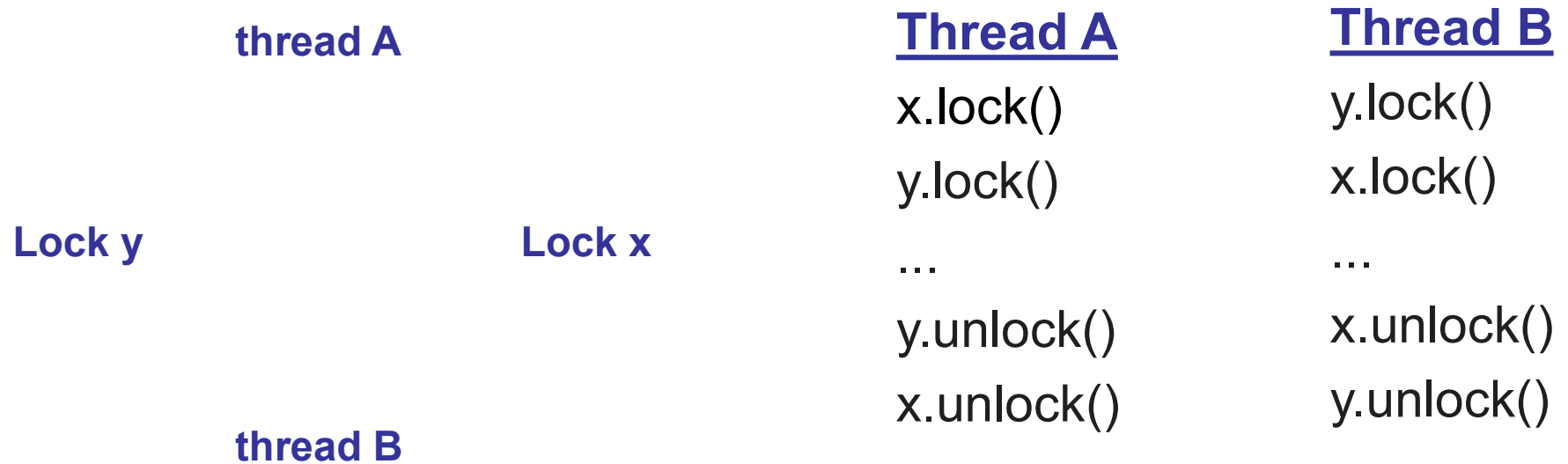
...

x.unlock

y.unlock

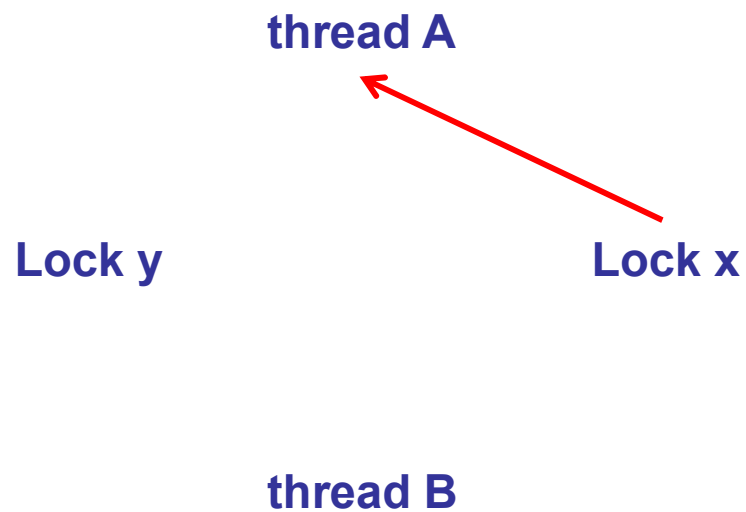
- Can deadlock occur?
- Will deadlock always occur?

Wait-for graph



Cycle represents a deadlock

Wait-for graph



Thread A

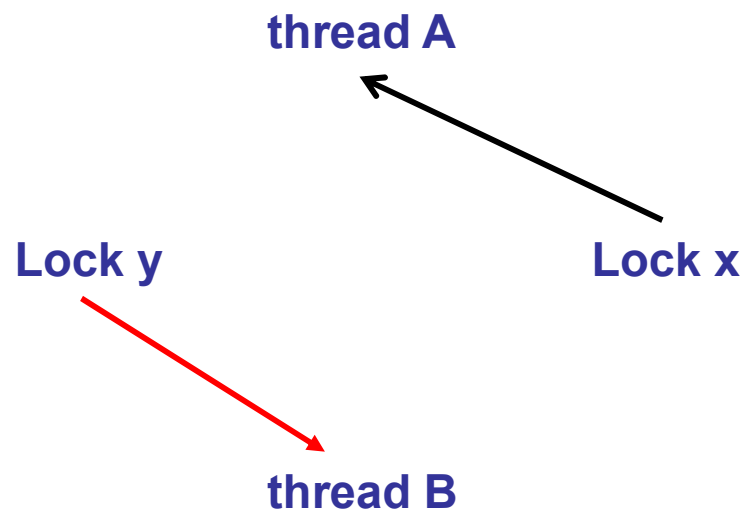
x.lock()
y.lock()
...
y.unlock()
x.unlock()

Thread B

y.lock()
x.lock()
...
x.unlock()
y.unlock()

Cycle represents a deadlock

Wait-for graph



Thread A

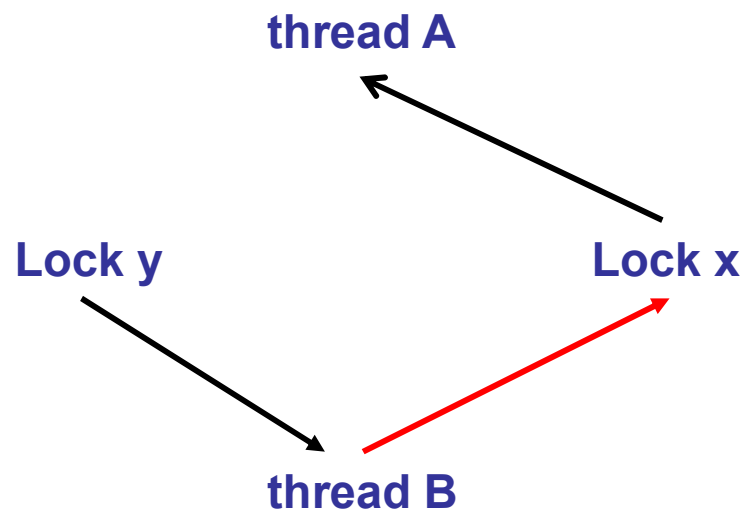
```
x.lock()  
y.lock()  
...  
y.unlock()  
x.unlock()
```

Thread B

```
y.lock()  
x.lock()  
...  
x.unlock()  
y.unlock()
```

Cycle represents a deadlock

Wait-for graph



Thread A

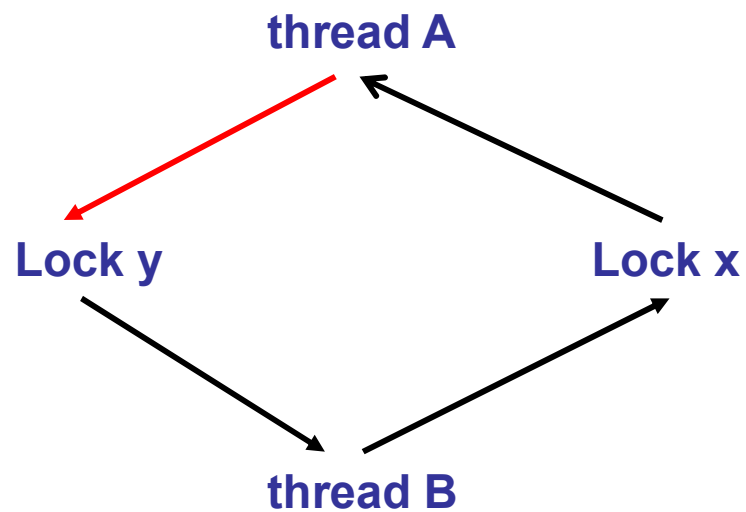
```
x.lock()  
y.lock()  
...  
y.unlock()  
x.unlock()
```

Thread B

```
y.lock()  
x.lock()  
...  
x.unlock()  
y.unlock()
```

Cycle represents a deadlock

Wait-for graph



Thread A

x.lock()
y.lock()
...
y.unlock()
x.unlock()

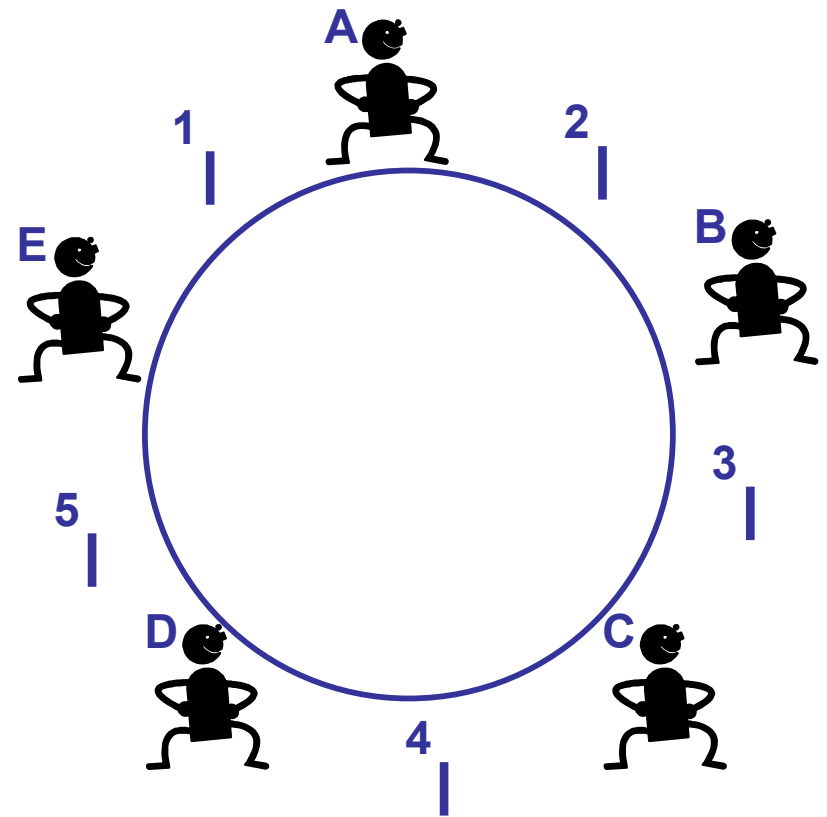
Thread B

y.lock()
x.lock()
...
x.unlock()
y.unlock()

Cycle represents a deadlock

Dining philosophers

- 5 philosophers sit at round table
- 1 chopstick between each pair of philosophers
- Each philosopher needs 2 chopsticks to eat



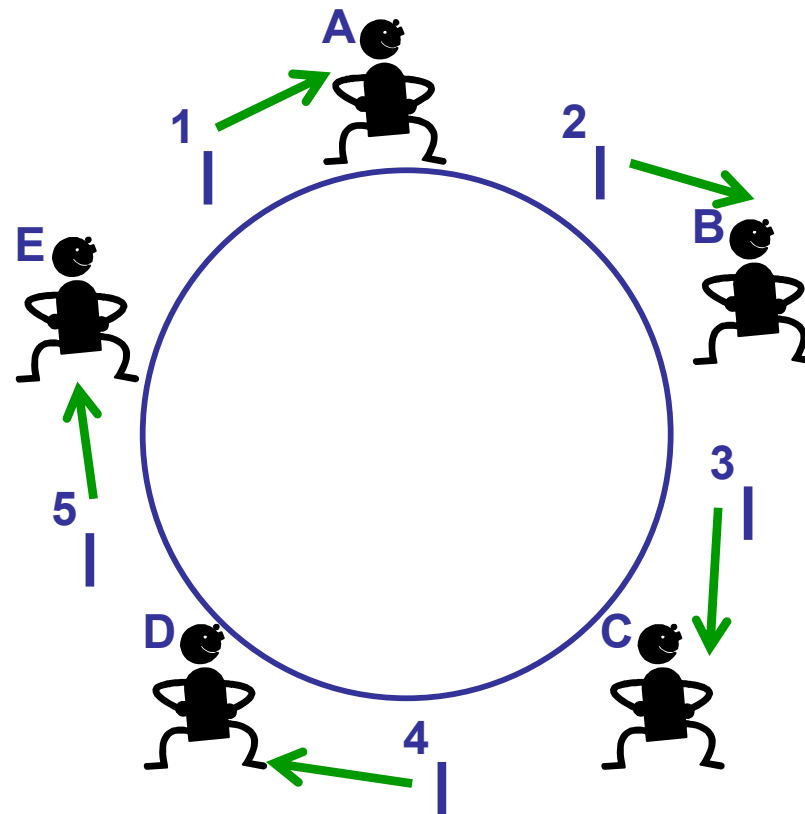
Dining philosophers

Algorithm for philosopher:

1. wait for chopstick on right to be free
2. pick up chopstick on right
3. wait for chopstick on left to be free
4. pick up chopstick on left
5. eat
6. put both chopsticks down

- Can this deadlock?

Dining philosophers



Generic example of multi-threaded program

phase 1

```
while (!done) {  
    acquire some resource  
    work  
}
```

phase 2

```
release all resources
```

What to do about deadlocks

- Ignore
 - Typical OS strategy for application deadlocks
 - Deadlocked threads consume no CPU time
- Detect and fix
 - Detect: look for cycle in wait-for graph
 - Fix #1
 - » Kill one of the threads involved in the deadlock
 - » What about broken invariants?
 - Fix #2
 - » Roll back actions of one of the threads, then try again
 - » This fixes broken invariants
 - » Is it always possible to roll back a thread?

What to do about deadlocks

- Ignore
- Detect and fix
- Prevent
 - Think about what conditions are necessary for deadlock to occur

Project 2

- Use lots of assertions
 - Assert any property that you expect to be true
 - Helps catch errors closer to where they occur
- Example: switch to null thread context. Would you rather:
 - Stop when you switch to the bad thread?
 - Stop when you put the bad thread onto the ready queue?
 - Stop when you put the bad thread onto a waiting queue?
- Example: At any time, a thread context must only be in one “place”
 - Where can a thread be?

Project 2: writing good test cases

- A good test suite provides comprehensive coverage
- Coverage along what dimension?
 - Code coverage
 - Specification coverage

Example: test if lock() blocks when mutex is held

Thread A

create thread B

m.lock()

m.unlock()

Thread B

m.lock()

m.unlock()

Writing good test cases

- Specification coverage
 - Targeted test cases (micro test case)
 - Stress test (macro test case)
- Which is better for debugging?

Four necessary conditions for deadlock

- **Limited resource**
 - Not enough to serve all threads simultaneously
 - Unlimited resource → no waiting for that resource
- **No preemption**
 - Can't force threads to give up resources
 - Preemption → no indefinite waiting
- **Hold and wait**
 - Hold resources while waiting to acquire others
 - No hold and wait → no multi-edge path in wait-for graph
- **Cycle of hold-wait requests**
 - No cyclical chain → no cycle in wait-for graph

How to eliminate limited resources?

- Increase # of resources
 - E.g., buy more machines
 - E.g., increase # of locks?

How to eliminate no preemption?

- Enable preemption
 - Preempt CPU
 - Preempt memory (next lecture topic)
 - Preempt lock?

How to eliminate hold and wait?

- First step:
 - Move resource acquisition to beginning

```
phase 1          all needed
                ×
while (!done) {
    acquire some resource
    work
}
```

```
phase 2
    release all resources
```

How to eliminate hold and wait?

- Second step
 - Option A: Acquire all needed resources **atomically**

phase 1

```
acquire all needed resources atomic  
while (!done) {  
    work  
}
```

phase 2

```
release all resources
```

Atomic acquisition

```
L.lock()
```

```
while right chopstick busy or left chopstick busy
```

```
    cv.wait (L)
```

```
pick up right chopstick
```

```
pick up left chopstick
```

```
L.unlock()
```

```
<eat>
```

```
L.lock()
```

```
drop left chopstick
```

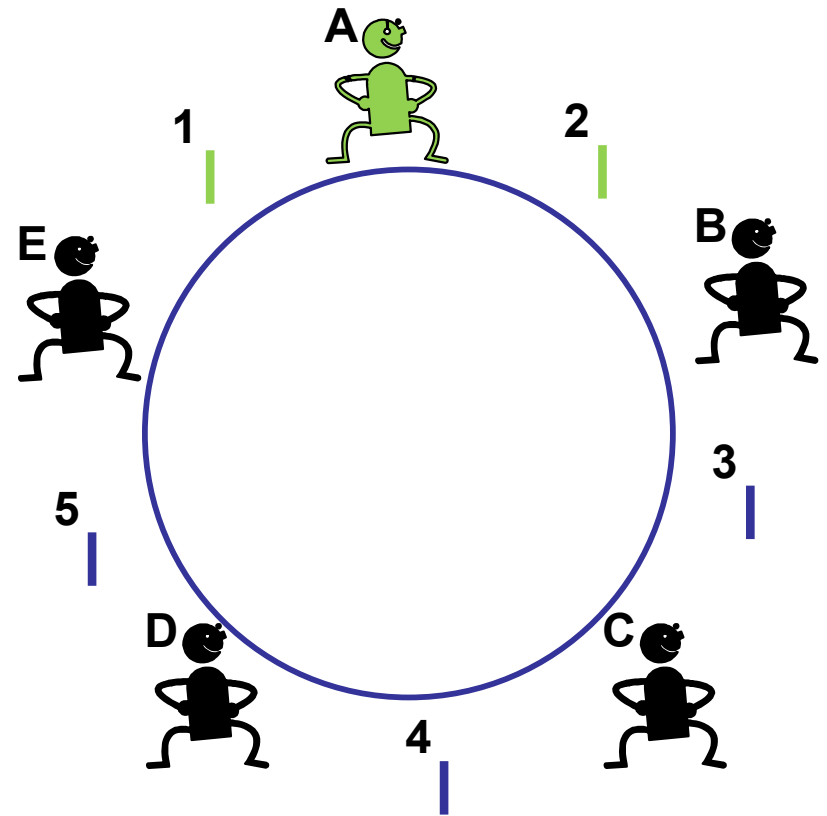
```
drop right chopstick
```

```
cv.broadcast()
```

```
L.unlock()
```

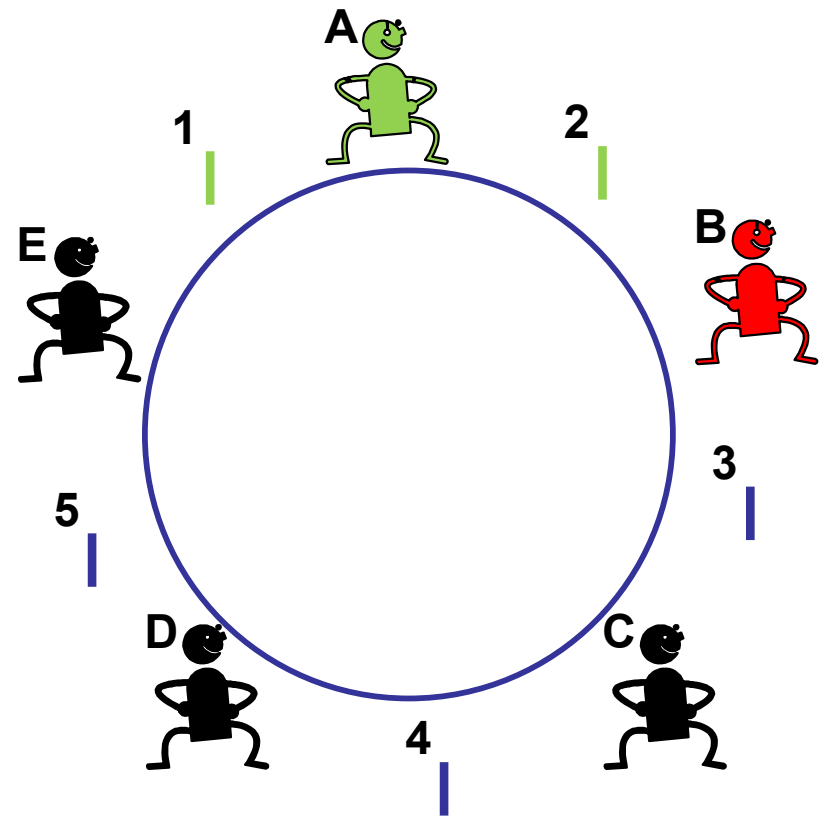
Dining philosophers

- A starts to eat



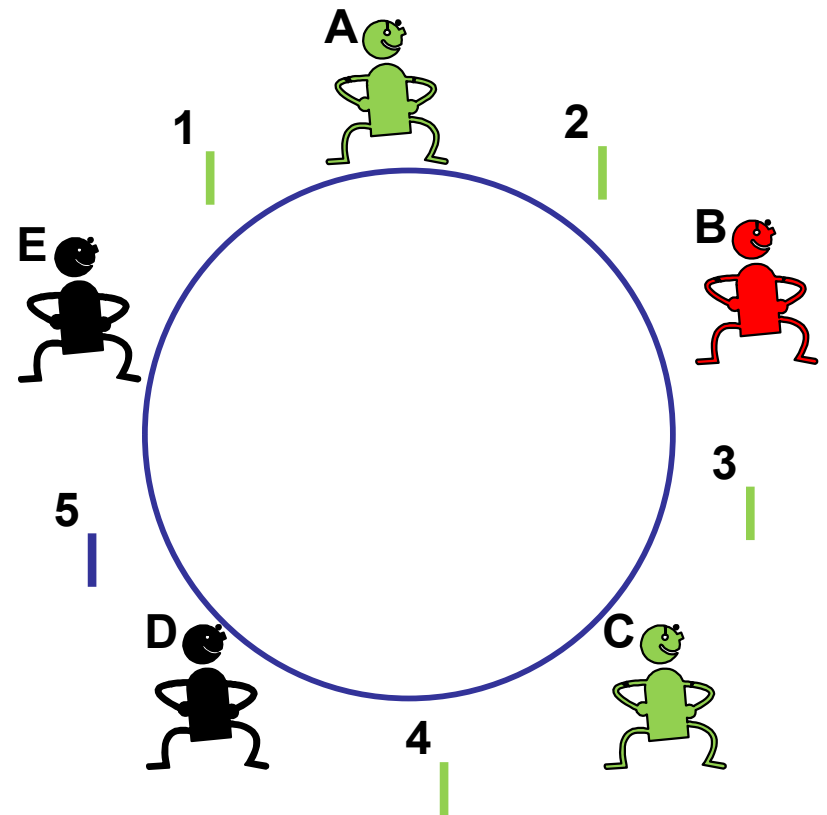
Dining philosophers

- A starts to eat
- B waits



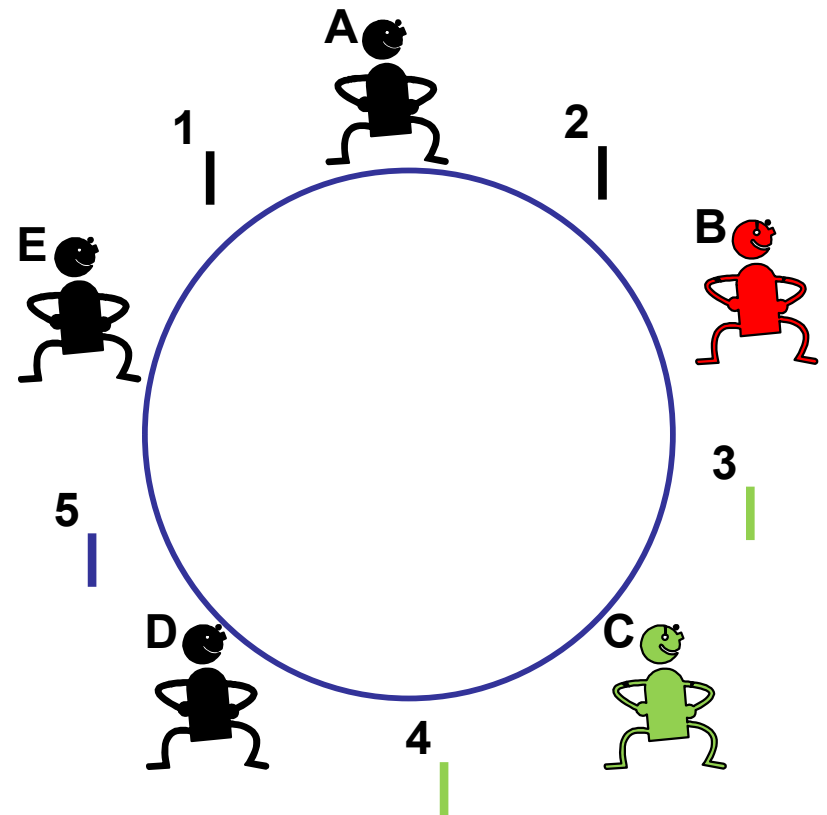
Dining philosophers

- A starts to eat
- B waits
- C starts to eat



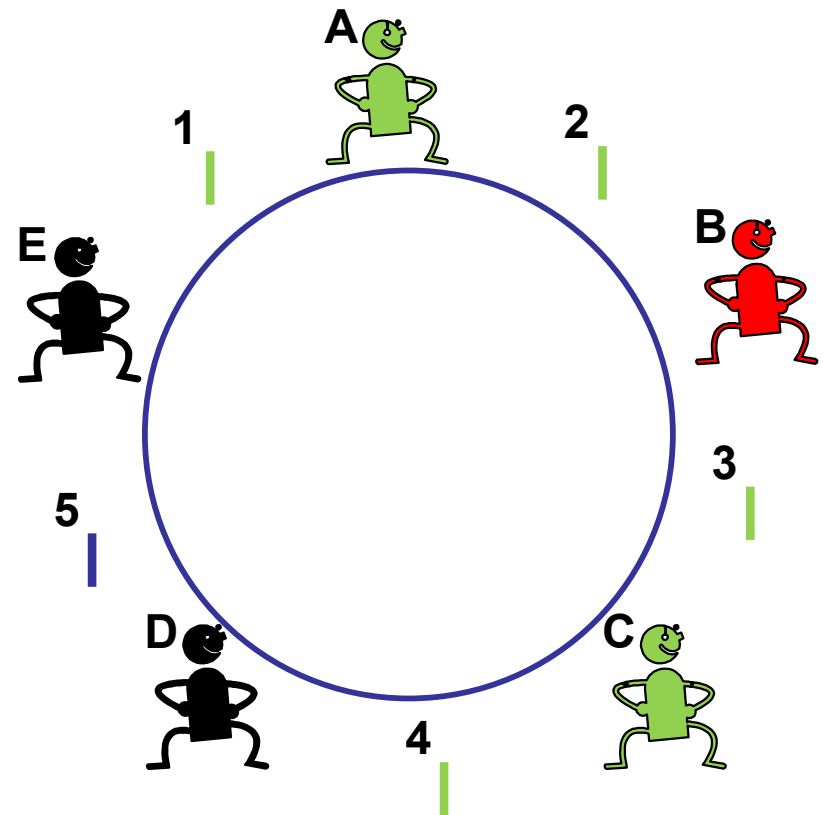
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes



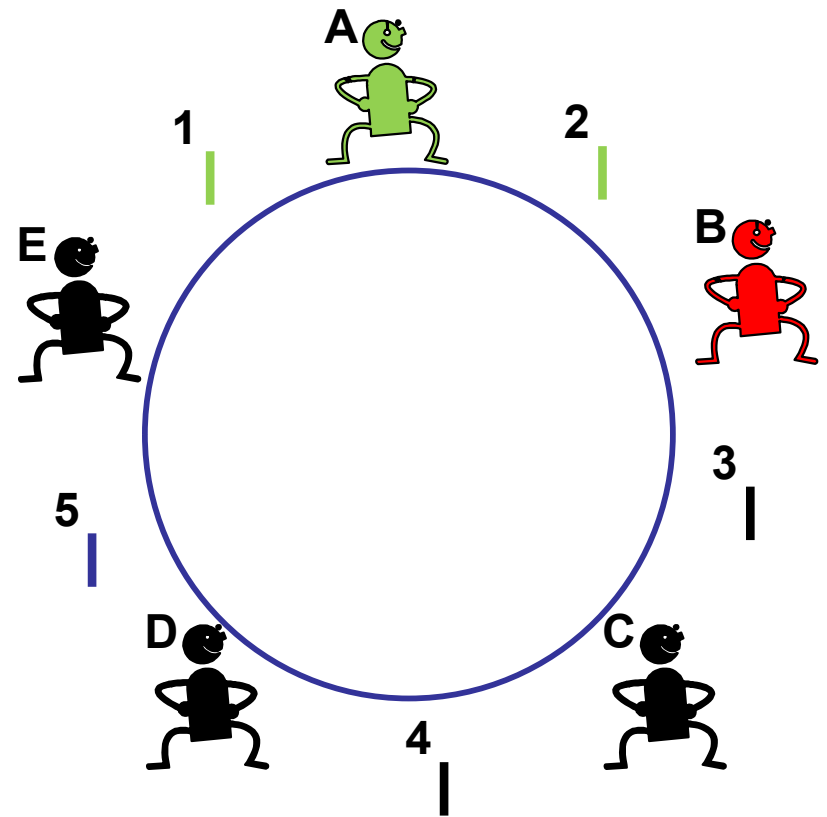
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat



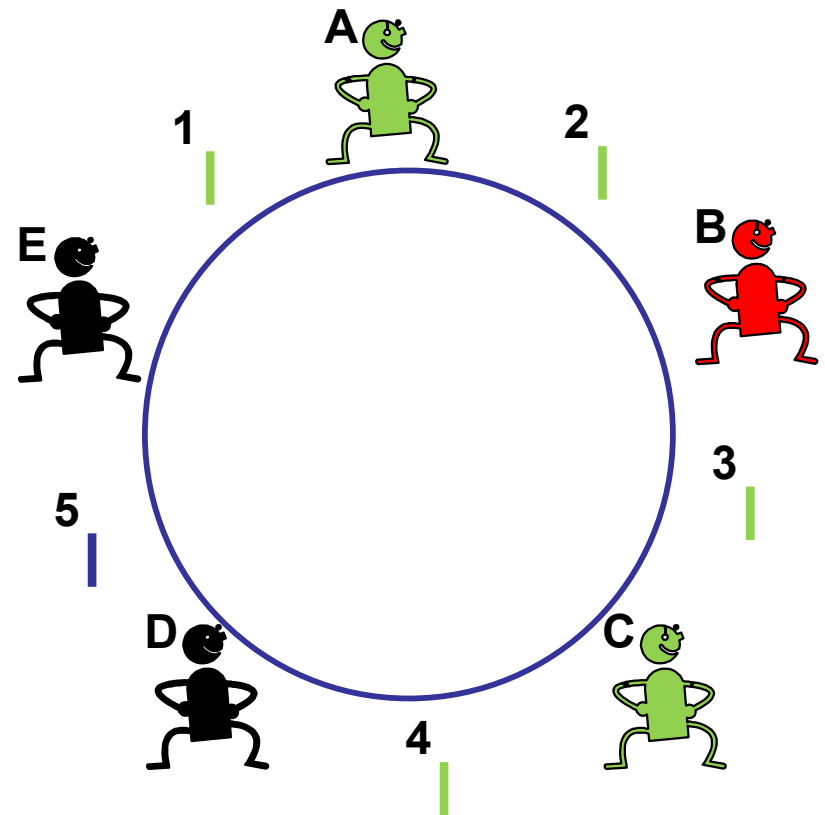
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes



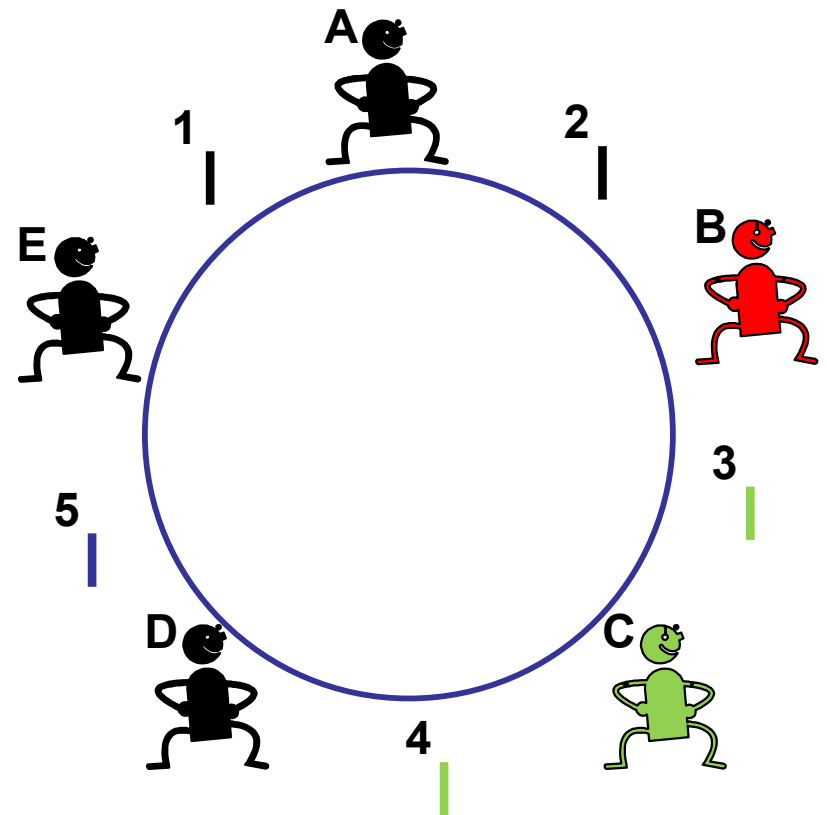
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat



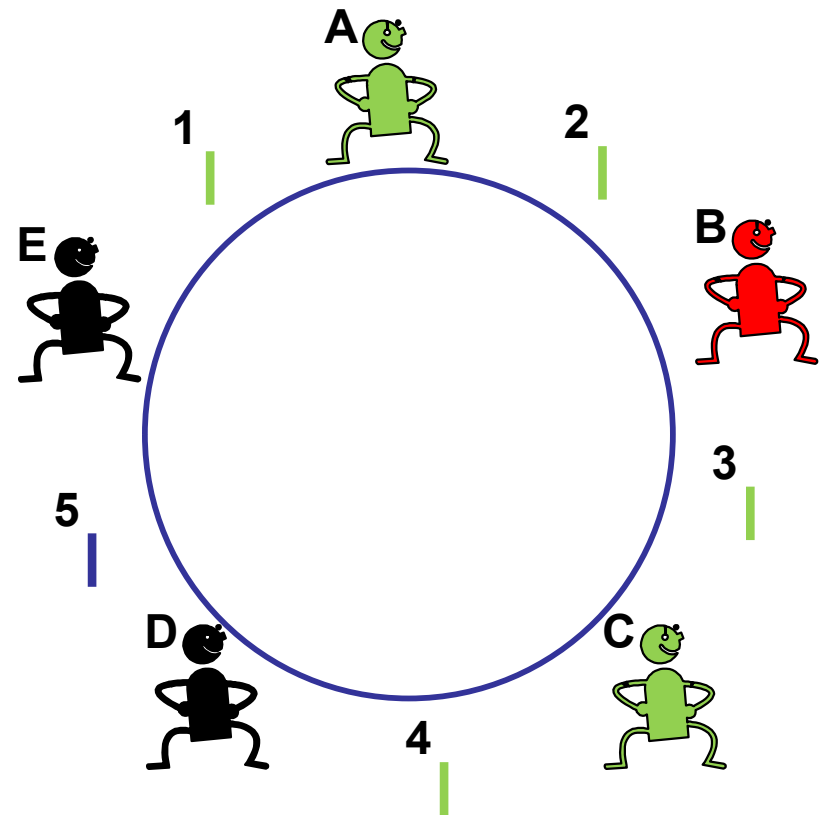
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat
- A finishes



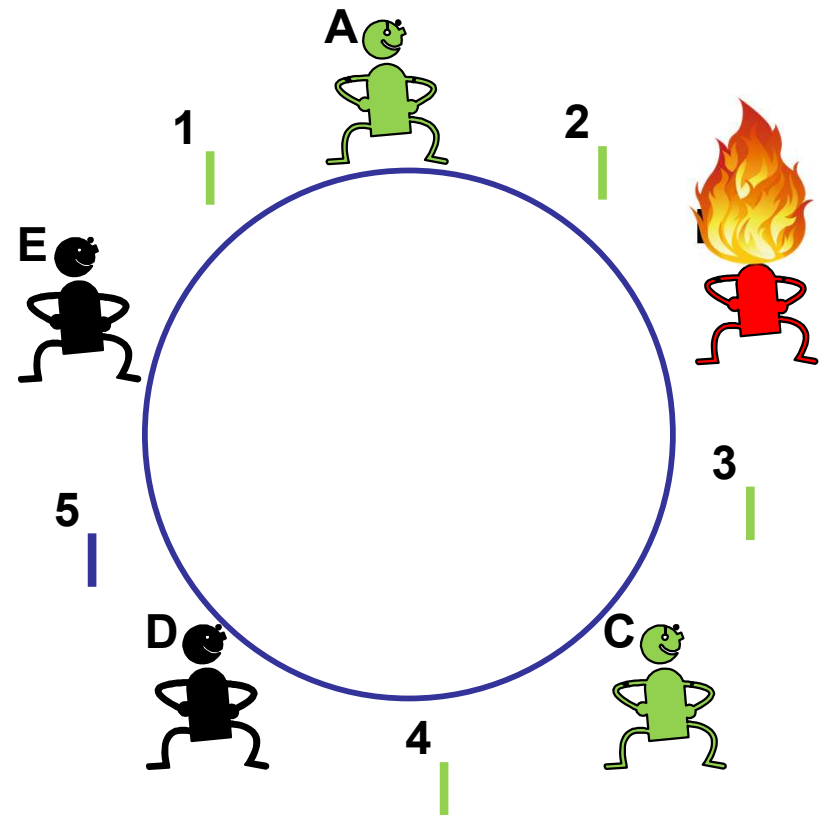
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat
- A finishes
- A starts to eat



Dining philosophers

- A starts to eat
- B waits
- while (1)
 - C starts to eat
 - A finishes
 - A starts to eat
 - C finishes



How to eliminate hold and wait?

- Second step
 - Option A: Acquire all needed resources **atomically**
 - Option B: Release and retry if you encounter a busy resource

phase 1

```
acquire all needed resources
while (!done) {
    work
}
```

phase 2

```
release all resources
```

How to eliminate hold and wait?

- What's the problem with this style of solution?

phase 1

```
acquire all needed resources
while (!done) {
    work
}
```

phase 2

```
release all resources
```

How to eliminate cycle of hold-and-wait?

- Define a global order over all resources
 - All threads **must** follow this order when acquiring resources
 - Guarantees that some thread can always make progress

Thread A

x.lock()

y.lock()

...

y.unlock()

x.unlock()

Thread B

y.lock()

x.lock()

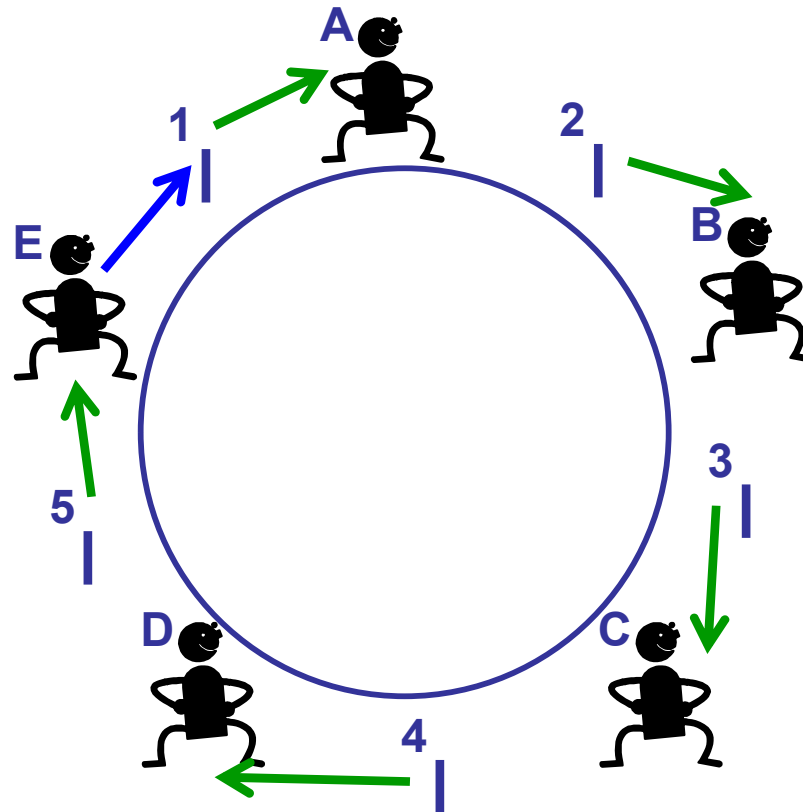
...

x.unlock()

y.unlock()

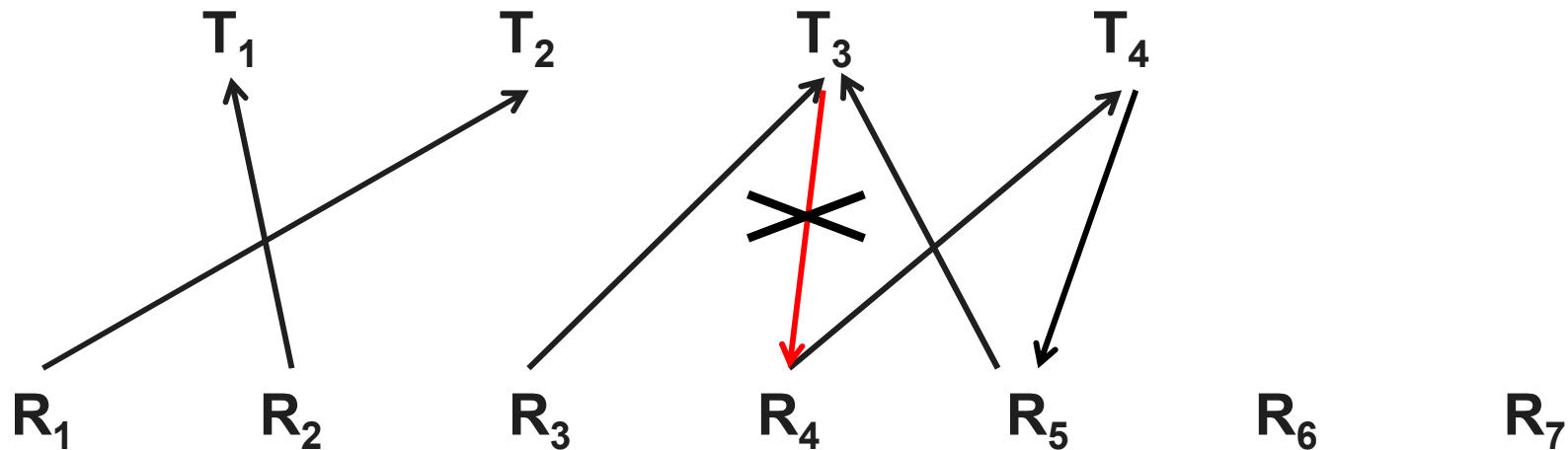
Dining philosophers

- Pick up lower # chopstick first
- Pick up higher # chopstick second



Global ordering of resources

- How can we be sure that *some* thread can make progress?



- Which thread can make progress?